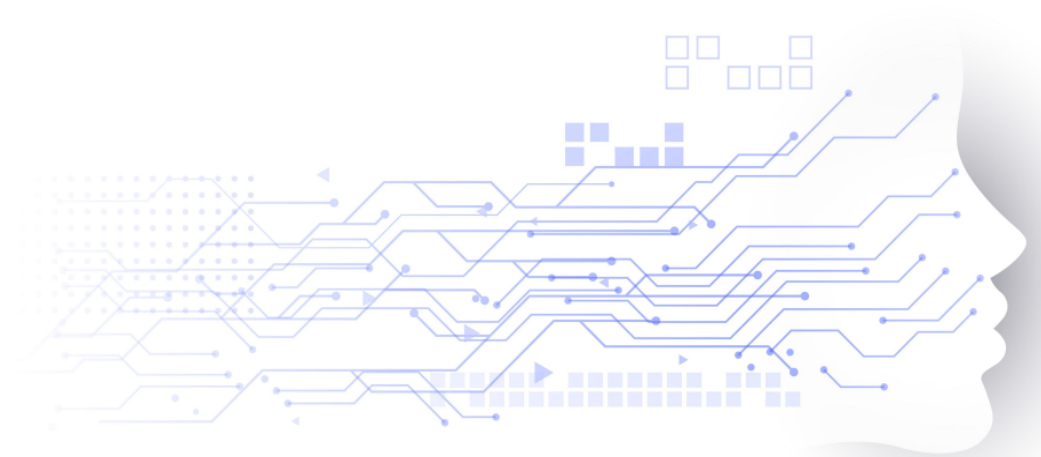




Decision Tree Learning

COSC2753 Machine Learning

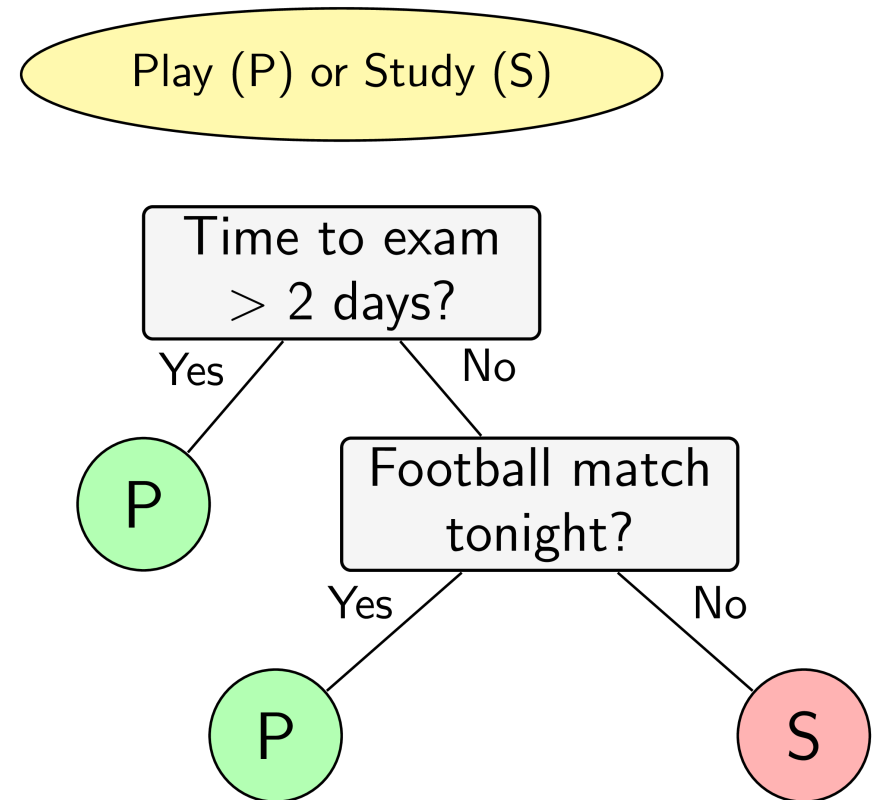
Semester 2, 2026

Decision Tree – Example

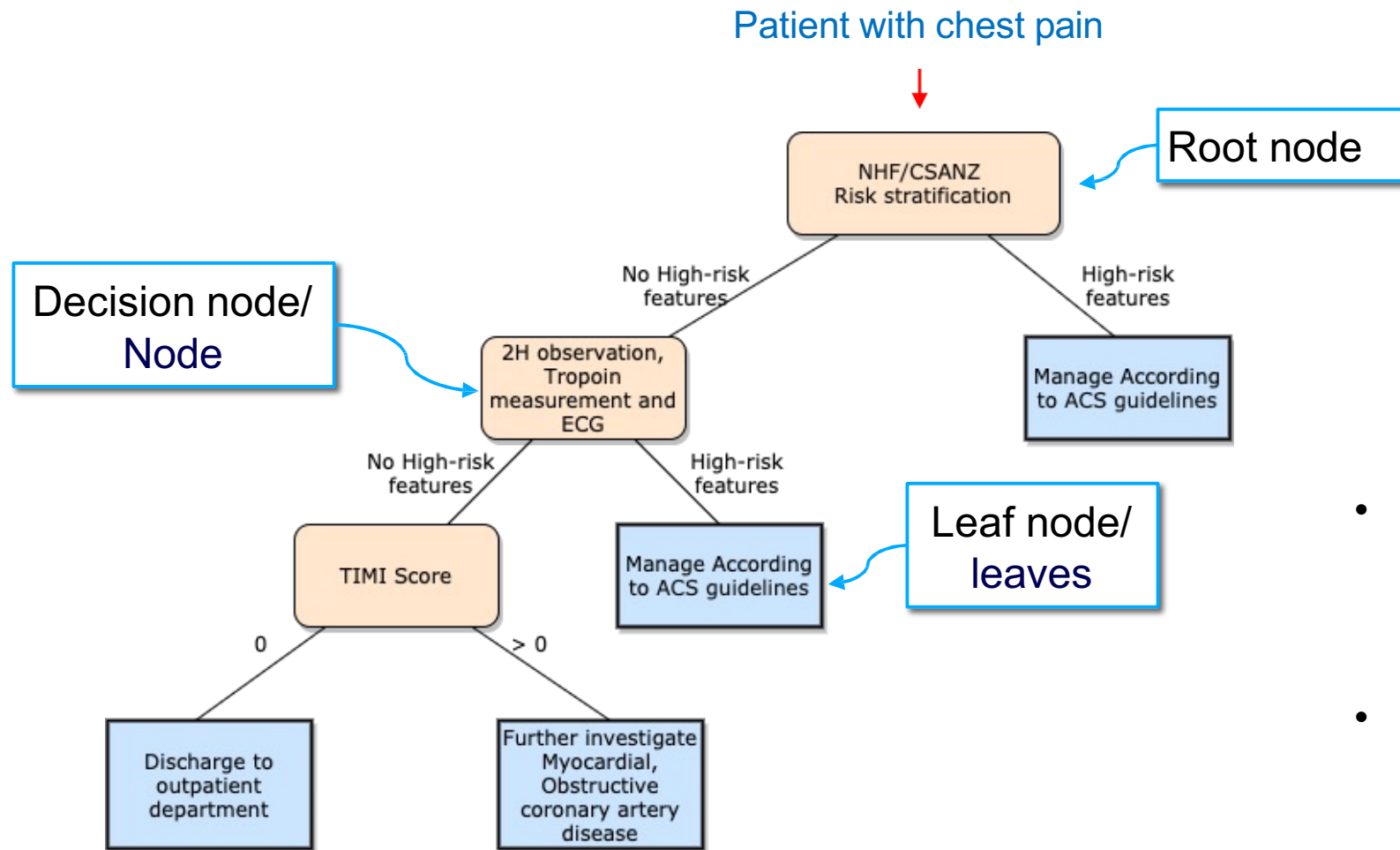
- Rules for students to decide

Play or Study:

- Time to exam > 02 days or not?
- There is a football match tonight?



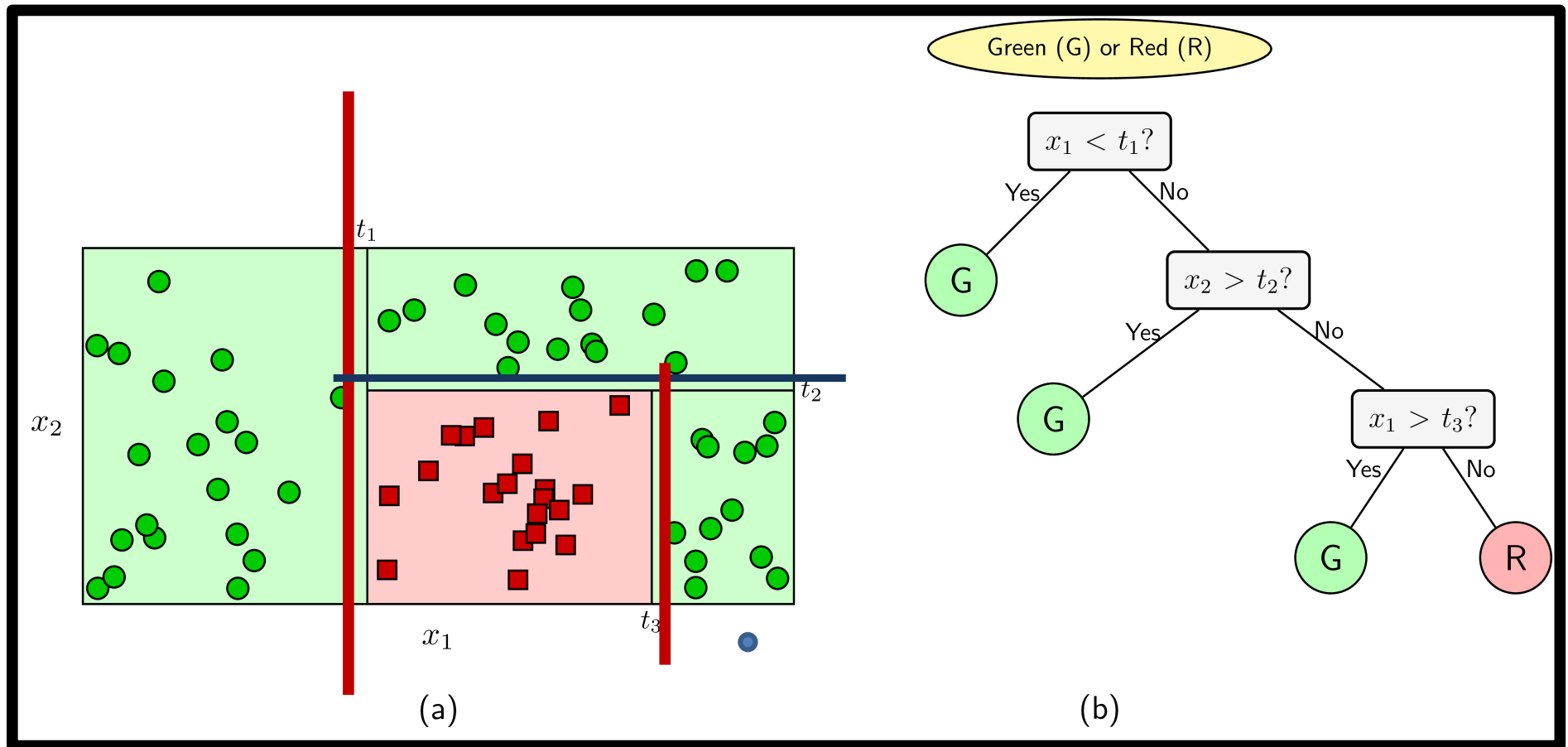
Decision Tree



- Does not have to perform all tests for each patient
- Easy to interpret and understand

Parsonage, W.A., Cullen, L. and Younger, J.F., 2013. The approach to patients with possible cardiac chest pain. *Medical Journal of Australia*, 199(1), pp.30-34.

Decision Tree – Example

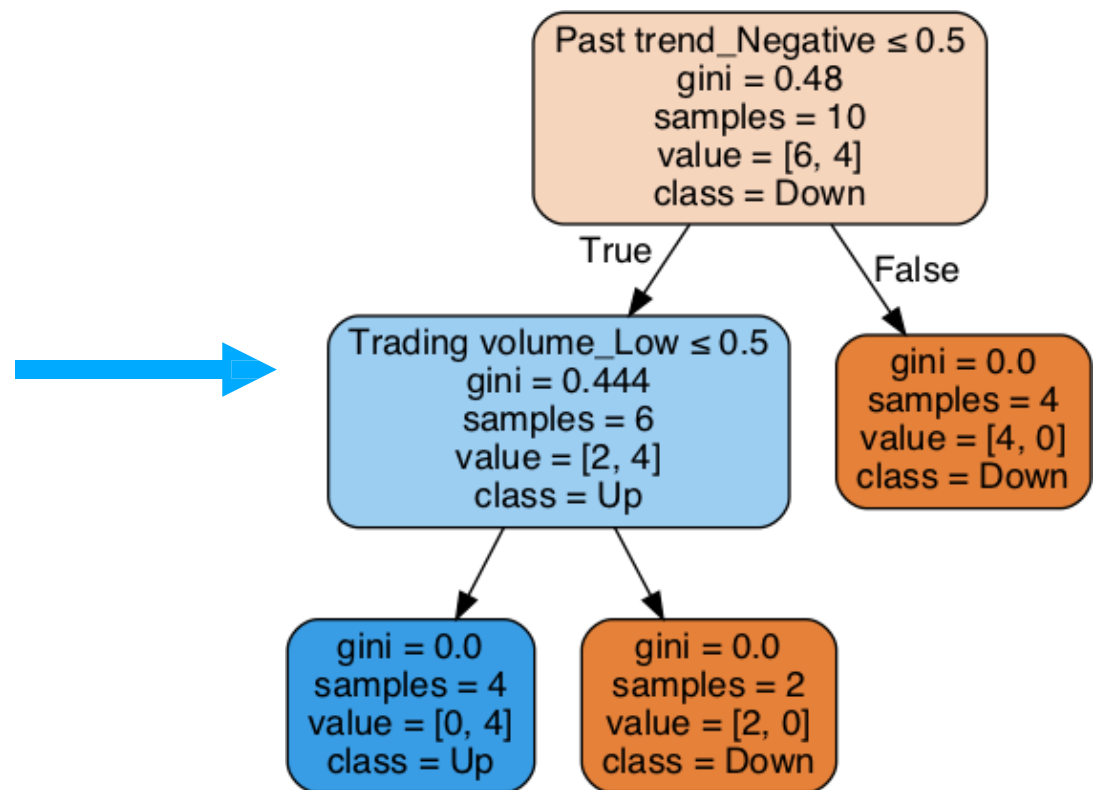


Decision Tree

How to predict?

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	Up
Negative	High	Low	Down
Positive	Low	High	Up
Positive	High	High	Up
Negative	Low	High	Down
Positive	Low	Low	Down
Negative	High	High	Down
Negative	Low	High	Down
Positive	Low	Low	Down
Positive	High	High	Up

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	High	High	?



Decision Tree

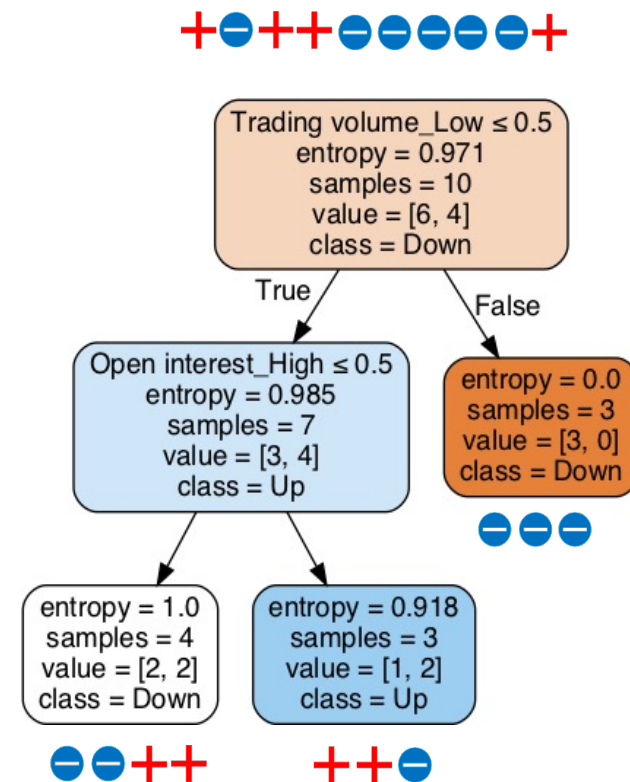
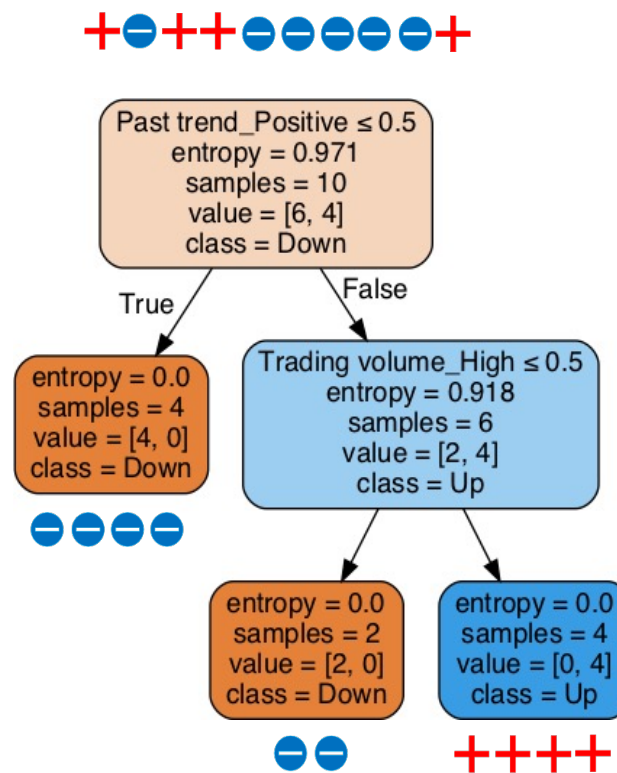
How to generate a tree given a dataset?

For now, let's assume the following:

- The features (attributes) are binary.
- The target variable is binary.

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	Up
Negative	High	Low	Down
Positive	Low	High	Up
Positive	High	High	Up
Negative	Low	High	Down
Positive	Low	Low	Down
Negative	High	High	Down
Negative	Low	High	Down
Positive	Low	Low	Down
Positive	High	High	Up

Objective (Loss) for Decision Trees



Entropy

Entropy is a measure of the **impurity** of a data set.

- An entropy of 0 implies a data set is pure
- The higher the entropy, the more impure a data set

Entropy is given by

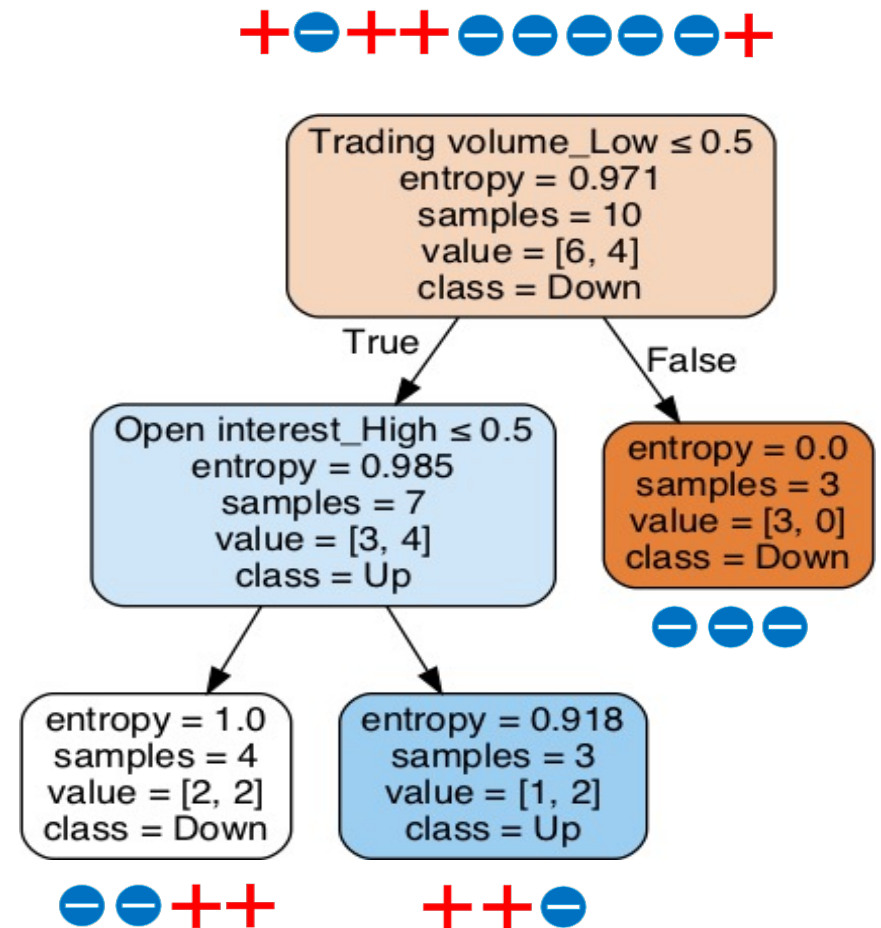
$$Entropy(S) = - \sum_{c=1}^c p(c) \log_2 p(c)$$

- S is the data set
- c is each class
- $p(c)$ is the proportion of S which have class c

Entropy

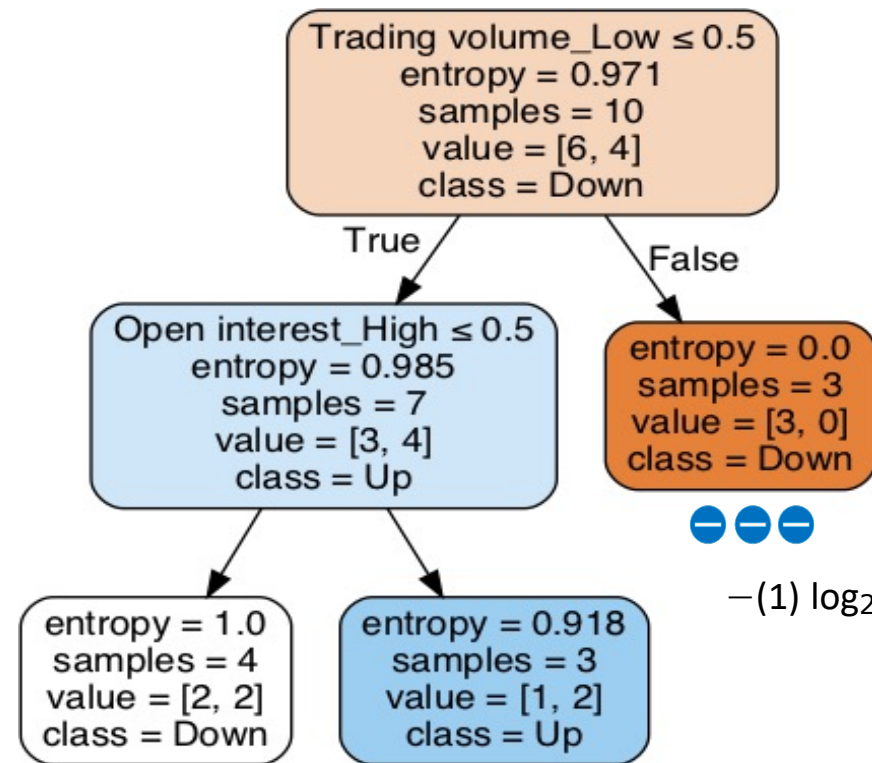
$$Entropy(S) = - \sum_{c=1}^c p(c) \log_2 p(c)$$

Entropy of each leaf node???



Entropy

$$Entropy(S) = - \sum_{c=1}^c p(c) \log_2 p(c)$$



— — —

$$-(1) \log_2(1) = 0.0$$

— — ++

++ —

$$-0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1.0$$

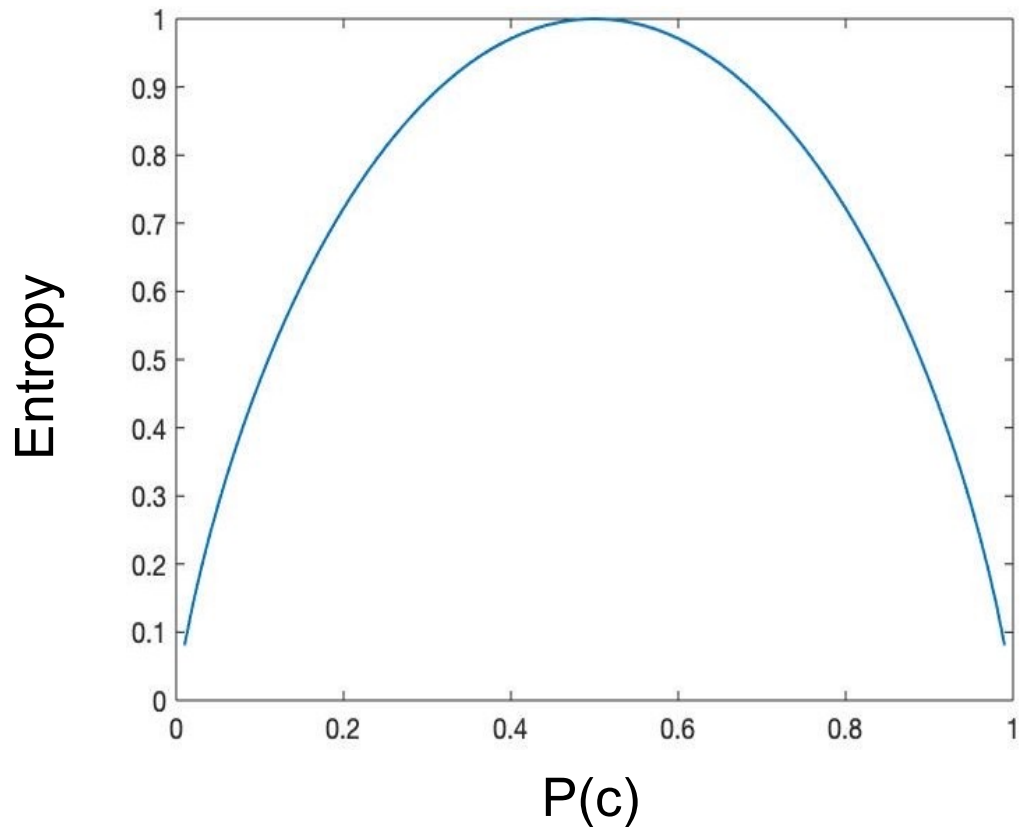
$$-(2/3) \log_2(2/3) - (1/3) \log_2(1/3) = 0.9183$$

Entropy

Consider an example data set with only positive ($\oplus$) and negative ($\ominus$) classes

Let $p_{\oplus}$ and $p_{\ominus}$ be the proportion of positive and negative examples

What does the entropy function look like?



$$Entropy(S) = - \sum_{c=1}^c p(c) \log_2 p(c)$$

Information Gain

How can entropy be used?

Given the current node, how to find the best split?

- Choose an attribute to split on that gives the largest decrease in entropy across all sub-trees (i.e., more consensus/purity)
- This is the effectiveness of the attribute for classifying the data

The **change/reduction in entropy** is called the **information gain** of the attribute

Information Gain

$$Gain(S, A) = Entropy(S) - \sum_{v \in values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- S is the data set
- A is a given attribute
- S_v is the subset of example from S such that the value of attribute $A = v$

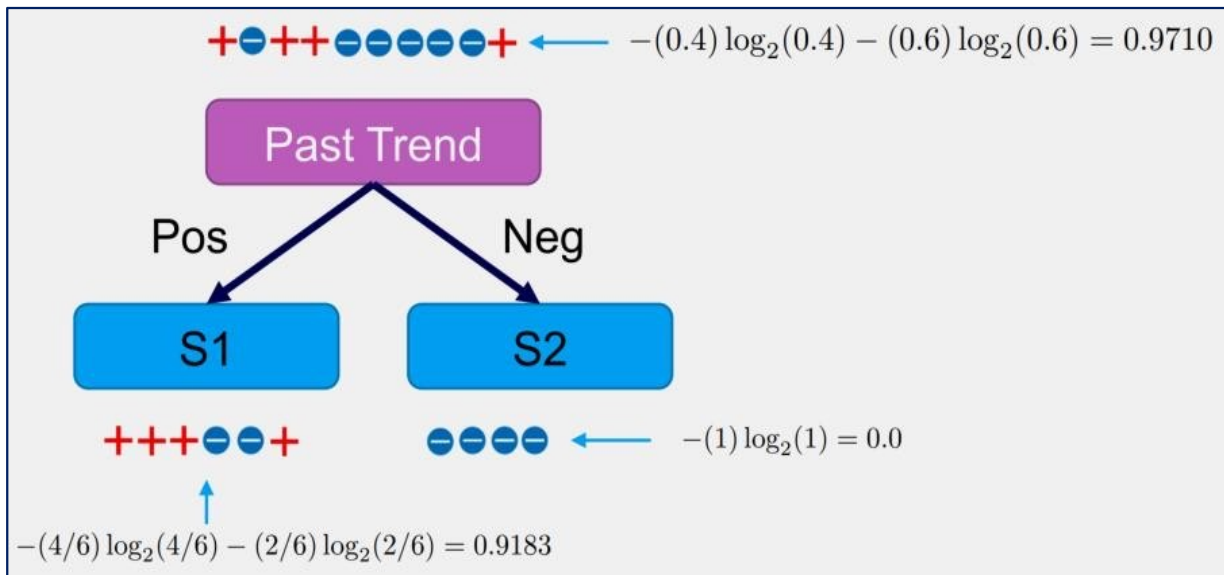
Information Gain

$$Gain(S, A) = Entropy(S) - \sum_{v \in values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- Attribute A^* is chosen for splitting if it **maximize the Gain**

Example

Gain after splitting $S \rightarrow S_1, S_2$

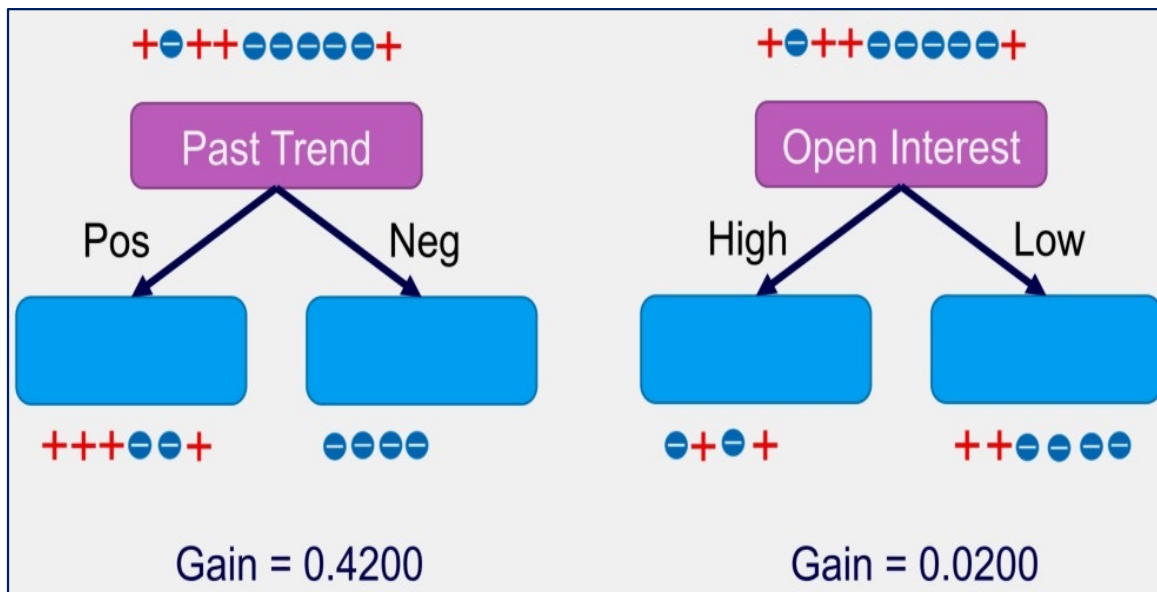


Trend	Interest	Volume	(Target)
Positive	Low	High	+
Negative	High	Low	-
Positive	Low	High	+
Positive	High	High	+
Negative	Low	High	-
Positive	Low	Low	-
Negative	High	High	-
Negative	Low	High	-
Positive	Low	Low	-
Positive	High	High	+

$$\begin{aligned}
 \text{Gain}(S, A) &= \text{Entropy}(S) - \frac{|S_1|}{|S|} \text{Entropy}(S_1) - \frac{|S_2|}{|S|} \text{Entropy}(S_2) \\
 &= 0.9710 - \frac{6}{10} 0.9183 - \frac{4}{10} 0.0 \\
 &= 0.4200
 \end{aligned}$$

Example

Gain after splitting $S \rightarrow S1, S2$

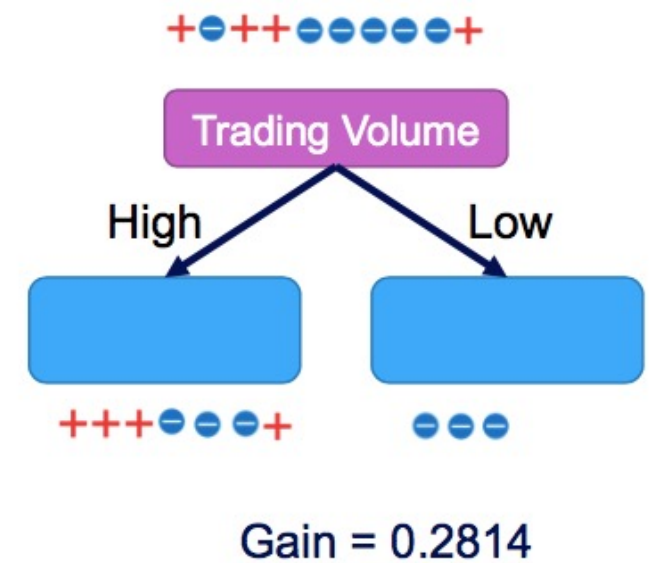
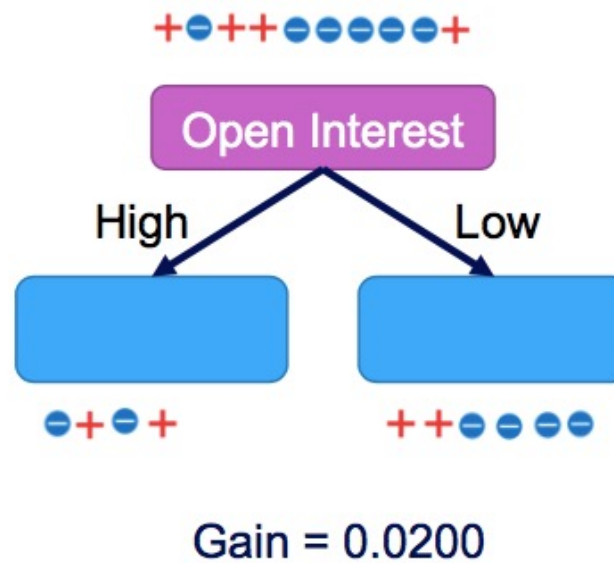
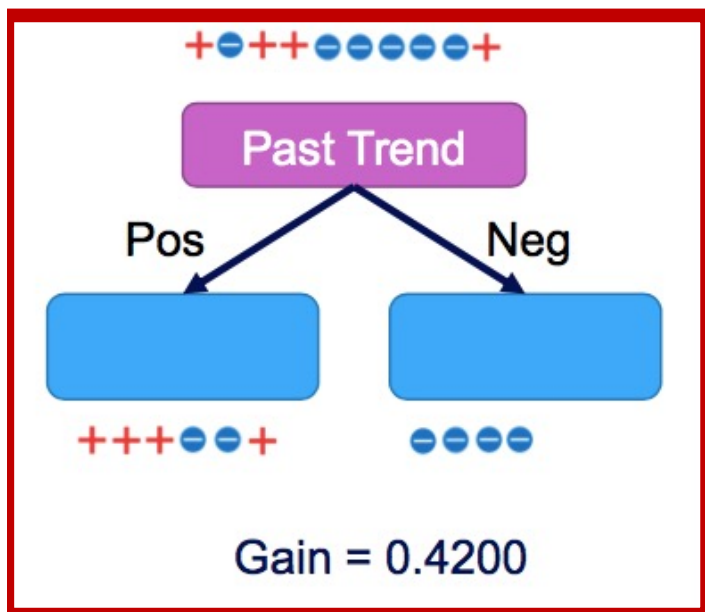


Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	+
Negative	High	Low	-
Positive	Low	High	+
Positive	High	High	+
Negative	Low	High	-
Positive	Low	Low	-
Negative	High	High	-
Negative	Low	High	-
Positive	Low	Low	-
Positive	High	High	+

Example

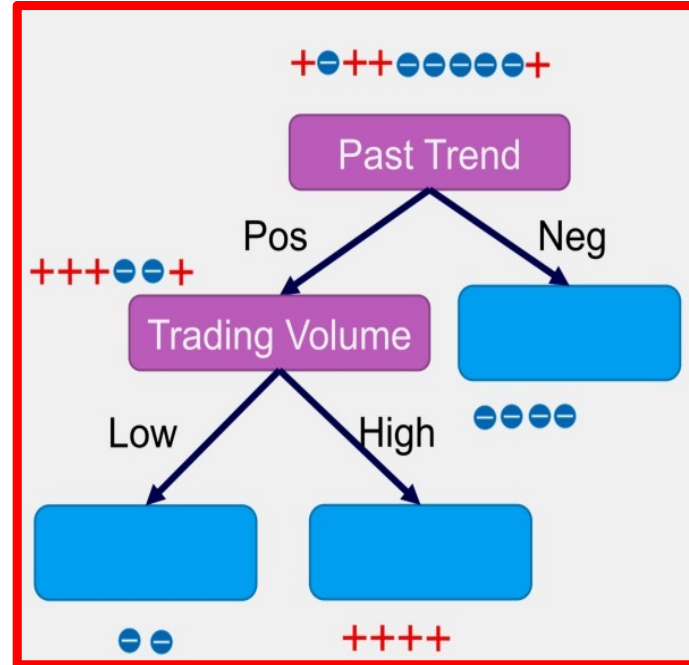
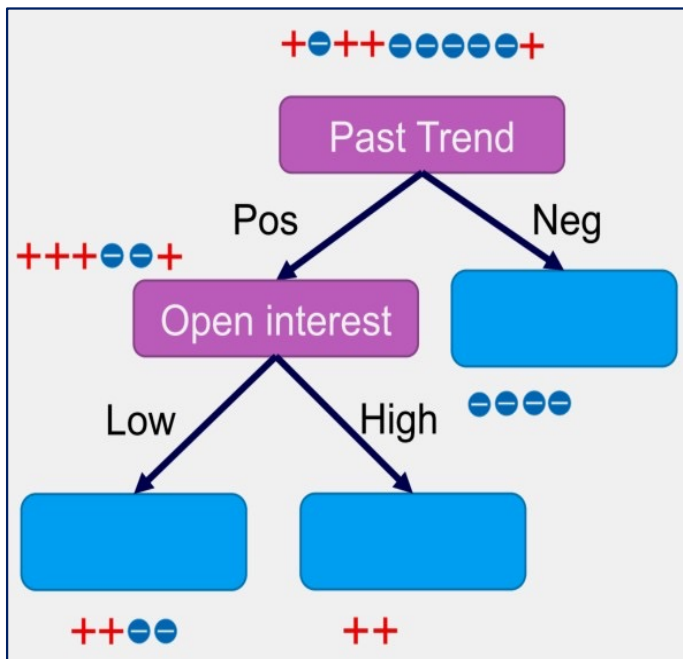
Gain after splitting $S \rightarrow S1, S2$

	Interest	Volume	Change
Positive	Low	High	+
Negative	High	Low	-
Positive	Low	High	+
Positive	High	High	+
Negative	Low	High	-
Positive	Low	Low	-
Negative	High	High	-
Negative	Low	High	-
Positive	Low	Low	-
Positive	High	High	+



Example

The averaged entropy of a split $S \rightarrow S1, S2$



Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	+
Negative	High	Low	-
Positive	Low	High	+
Positive	High	High	+
Negative	Low	High	-
Positive	Low	Low	-
Negative	High	High	-
Negative	Low	High	-
Positive	Low	Low	-
Positive	High	High	+

Decision Trees

Greedy Learning Algorithm **ID3**



Greedy Learning Algorithm (ID3)

`Split(Node, {examples}):`

1. $A \leftarrow$ The **"Best"** Attribute for splitting {examples}
2. Decision Attribute for this node $\leftarrow A$
3. For each value of A , create new child node
4. Split training {examples} to $child_nodes \leftrightarrow \{subset\}$
5. For each $child_node$
 - if subset is pure: STOP
 - else: `split(child_node, {subset})`

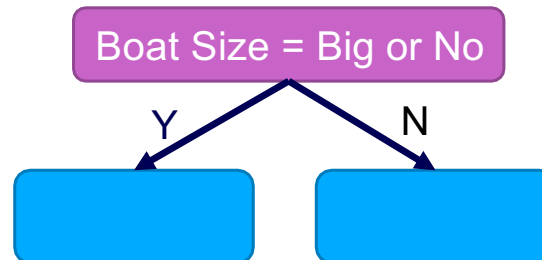
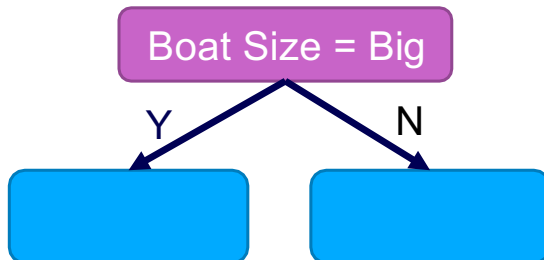
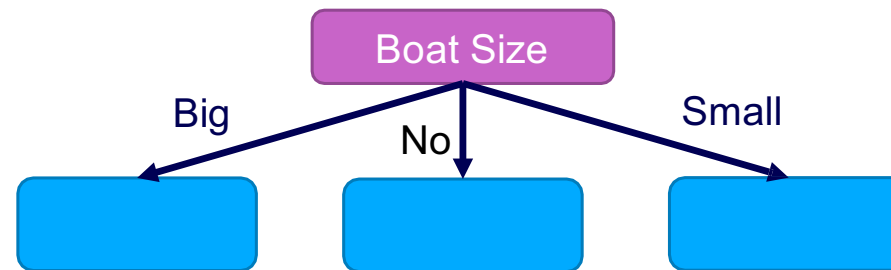
"Best" can be decided by: Information gain, **Gini Index** - [link](#)

$$\text{Gini}(S) = 1 - \sum_{c=1}^C p(c)^2$$

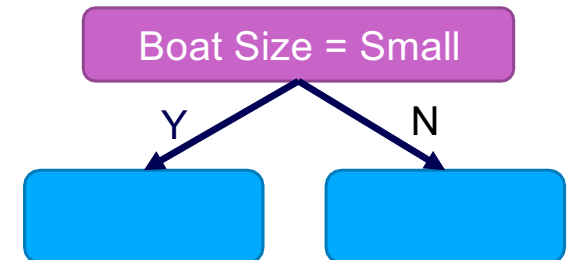
Attribute Types

Other types of Attributes

- Binary
- **Categorical**
- Ranked
- Real Values

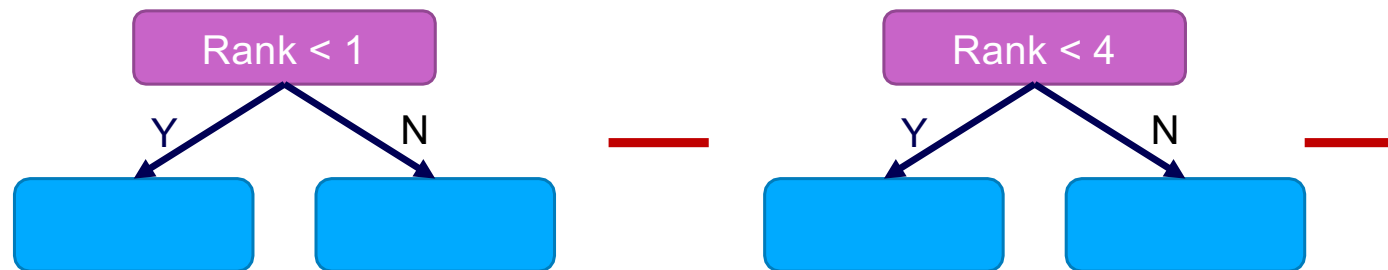


—



Other types of Attributes

- Binary
- Categorical
- **Ranked**
- Real Values



Other types of Attributes

- Binary
- Categorical
- Ranked
- **Real values**

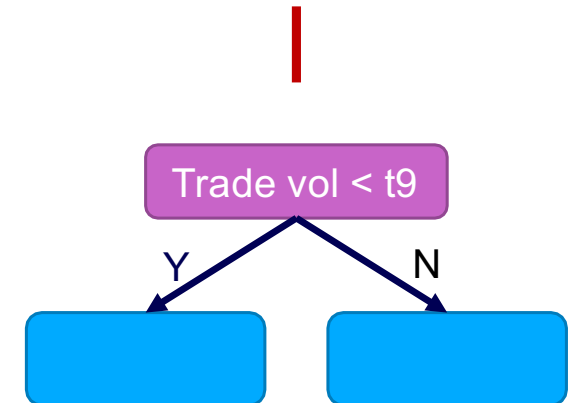
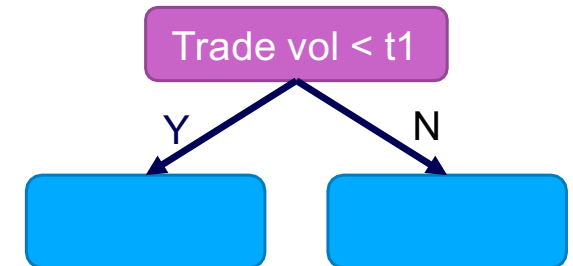
Sort Table with Attribute

Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	102	+
Negative	High	105	-
Positive	Low	125	+
Positive	High	133	+
Negative	Low	145	-
Positive	Low	167	-
Negative	High	180	-
Negative	Low	220	-
Positive	Low	250	-
Positive	High	300	+

← t1 = 103.5

← t2 = 115.0

← t9 = 275.0





Issues & Solutions



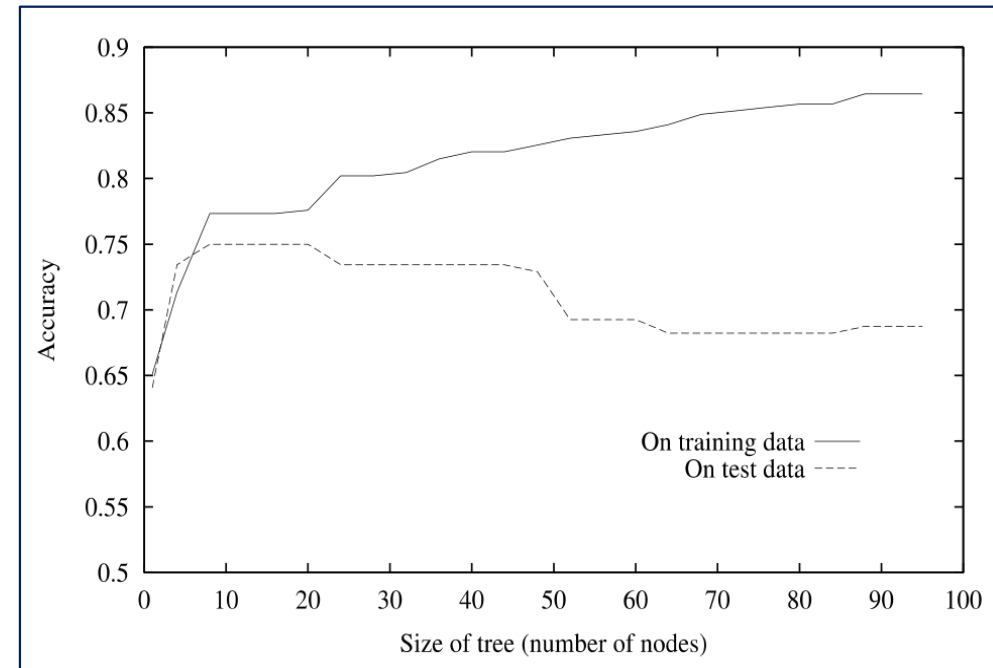
Issues in Decision Tree Learning

Homogeneous classification

- All leaves are pure, i.e., have an entropy of 0
- Leads to **over-fitting** of the training data

Noise or errors in the training data

- Noise will be still be separated and classified, as the tree will over-fit to the noise
- May result in a hypothesis that performs worse on test data



Pre-Pruning

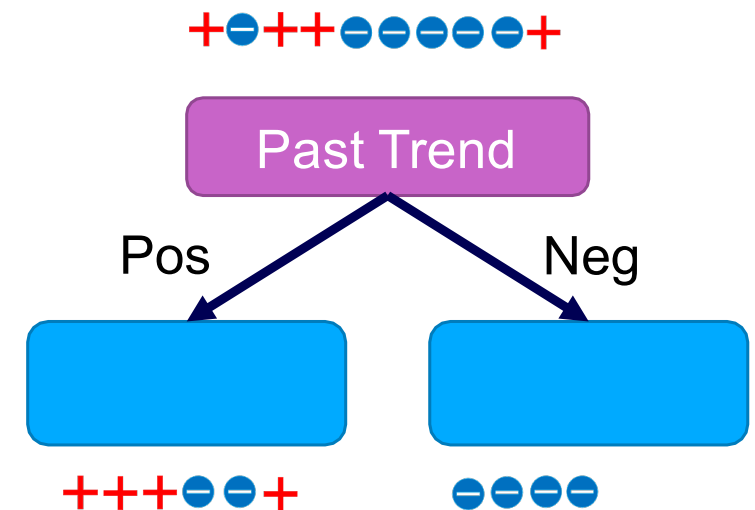
Stop generating sub-trees based on **properties** of the remaining training data:

- Minimum number of training examples (e.g., if 6 samples then stop splitting)
- Minimum impurity (or entropy)
- Minimum information gain
- Chi-squared test – only select statistically significant attributes

The classification of a leaf node: **majority class**

Comments on pre-pruning:

- Good for handling noise and errors
- Efficient to implement
- Susceptible to choice of attribute selection



Post-Pruning: Reduced Error Pruning

Post-pruning:

1. Set aside a percentage of the training data, as the **validation set**
2. Learn a decision tree, using pre-pruning
3. **Remove a sub-tree** that leads to leaves, and test the tree's error rate **using the validation data**
4. If the tree produces the **same or a better error**/classification rate, delete the sub-tree
5. Repeat until pruning sub-trees makes the classification rate worse



Reduced Error Pruning is similar to ***cross-validation***

C4.5 Algorithm

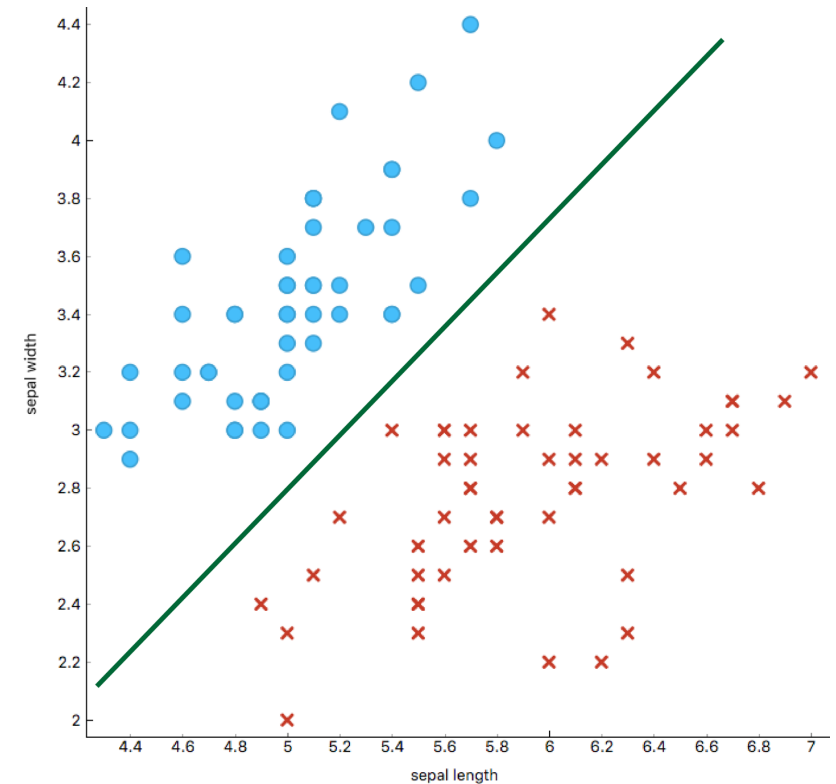
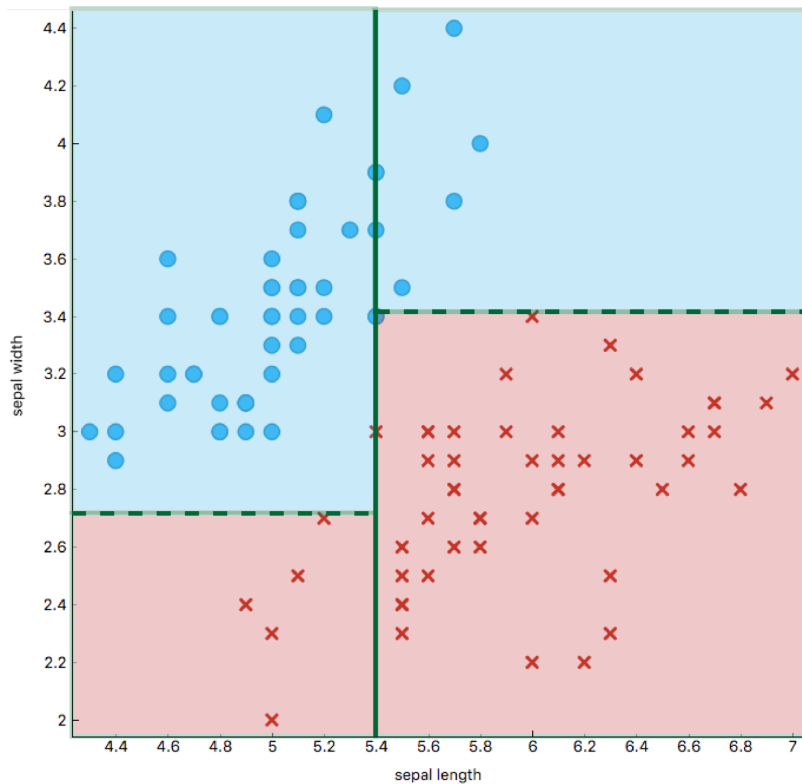
Famous and popular algorithm that addresses many issues with ID3

- Pruning (pre/post)
 - To address over-fitting
 - Helps to reduce the impact of noise
- Different splitting criteria – **Gain Ratio**
 - To address the inductive bias of information gain
- Can handle numeric (or continuous) valued attributes

Multi-variable splits

So far, these algorithms : **univariable**, considering only one attribute when splitting

This is sufficient, but can lead to less efficient splits



Multi-variable splits

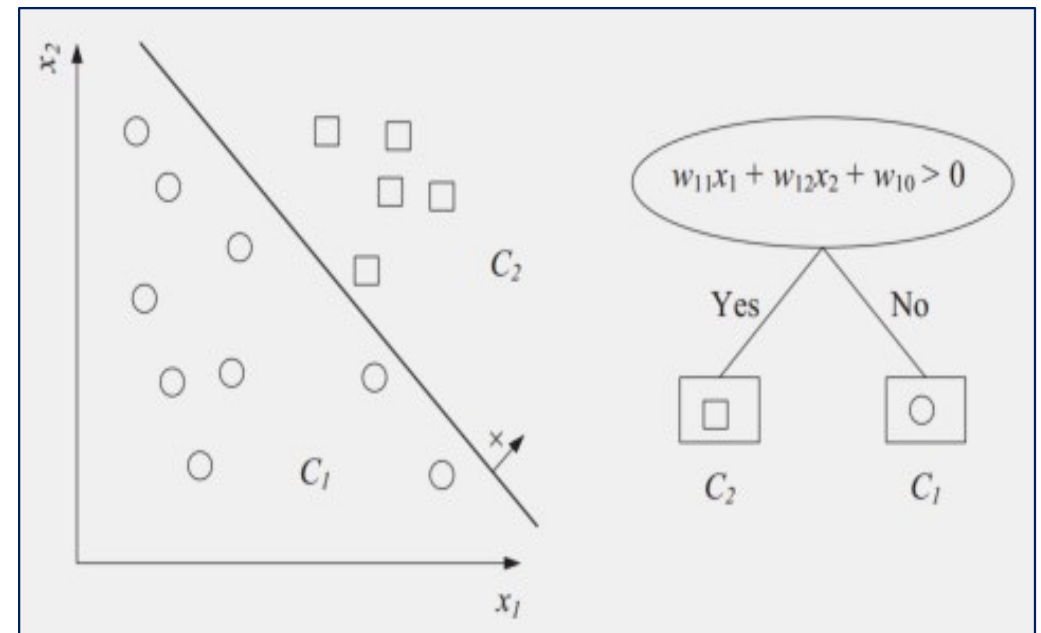
A multi-variable split, in the **form of a linear equation***, is more general

Takes the form:

$$\mathbf{w}_m^T \mathbf{x}_0 + w > 0$$

where $\mathbf{w}_m^T$ is the vector of weights

Requires all attributes to be numeric



CART was the original algorithm for generating multi-variable trees

Generating the linear equation is not cheap

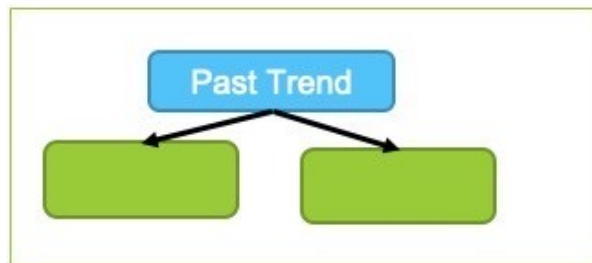
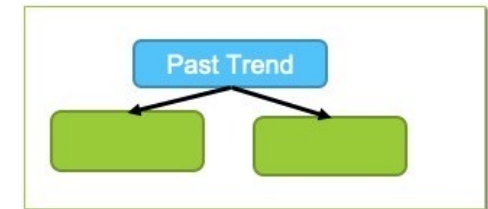
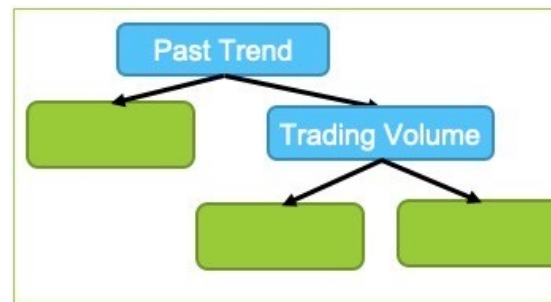
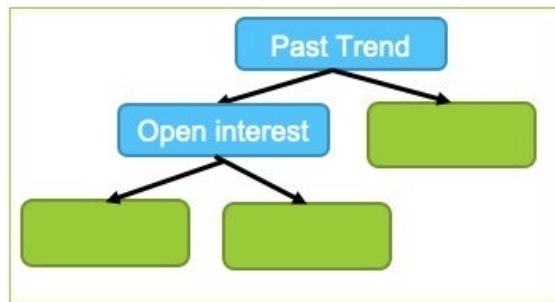
(*) doesn't have to be linear



From Trees to Forests



Random Forest

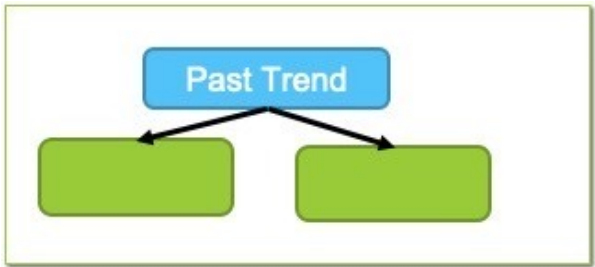
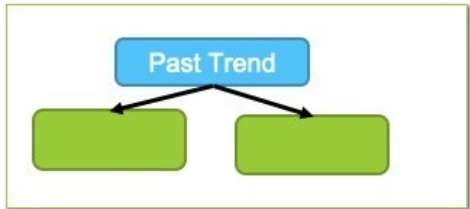
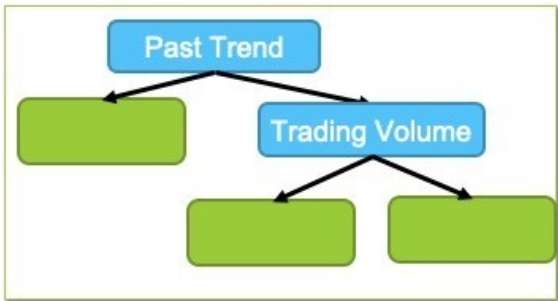
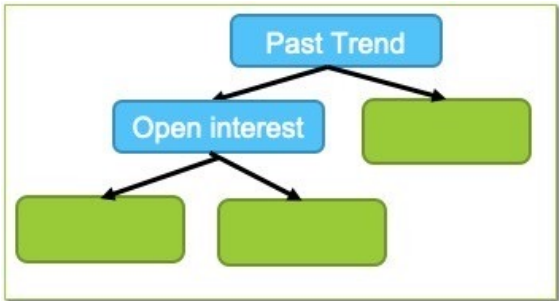


—



Random Forest - Testing

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	?



+	-
12	8



Bagging

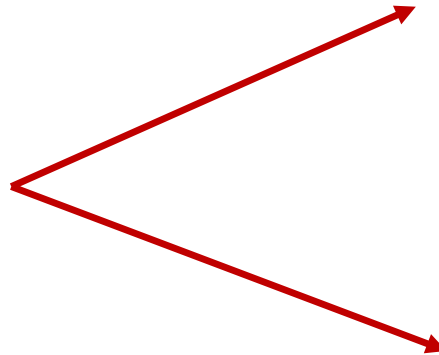
Bagging = “**B**ootstrap **A**ggregation”

- A simple way of voting or averaging over multiple learners
- In practice, provides improved performance for **unstable** learners
 - Learners which provide significantly different models if a small subset of the data is changed
 - Examples include Decision Trees
- Helps to reduce the impact of noise

How to make Multiple decision trees

Make a boot-strapped datasets

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	+
Negative	High	Low	-
Positive	Low	High	+
Positive	High	High	+
Negative	Low	High	-
Positive	Low	Low	-
Negative	High	High	-
Negative	Low	High	-
Positive	Low	Low	-
Positive	High	High	+

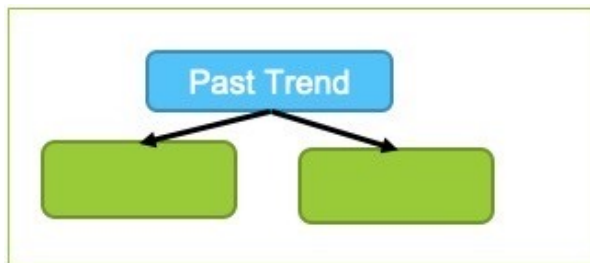
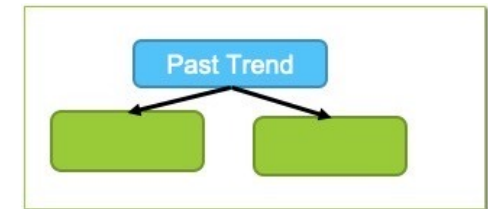
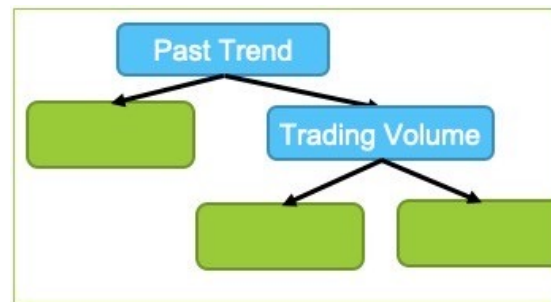
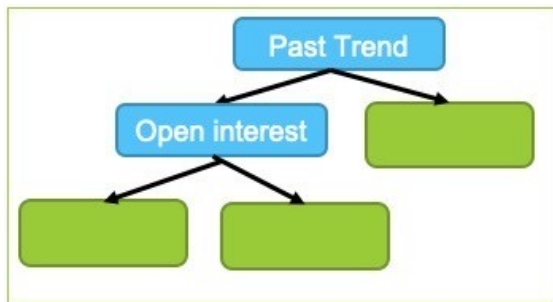


Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	+
Negative	High	Low	-

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	+
Negative	High	Low	-
Positive	Low	High	+
Negative	High	High	+
Positive	Low	High	-
Negative	Low	High	-
Positive	High	High	-
Negative	Low	Low	-
Positive	Low	Low	-
Negative	High	High	-
Negative	Low	High	-
Positive	Low	Low	-
Positive	High	High	+

- Draw a sample of size N from the original dataset **with replacement** → one bootstrap sample.
- Repeat step 1 S times, so that we have S bootstrap samples.

Random Forest

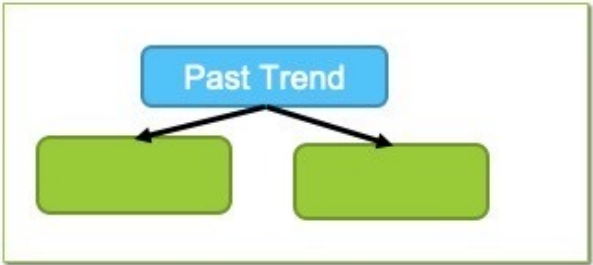
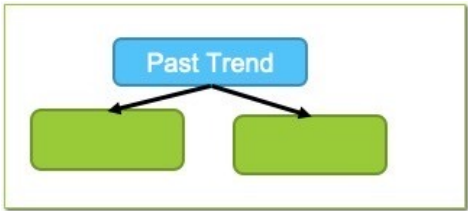
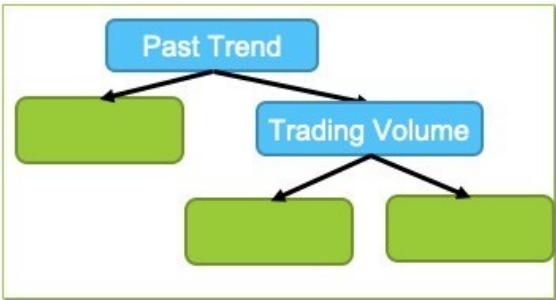
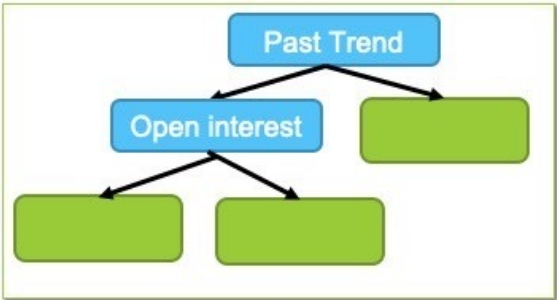


—



Random Forest - Testing

Past Trend	Open Interest	Trading Volume	Return (Target)
Positive	Low	High	?



+	-
12	8



Random Forest

Random Forest combines Decision Trees, Bagging, and Feature Bagging

- Trains a set of expert decision trees using:
 - Normal Bagging to construct the training data set
 - Feature Bagging to choose **k features** (at random) for each bagged data set.
- Each expert tree is deliberately kept small, by choosing $\sqrt{attributes}$ for feature bagging
- Final decision: **major voting**



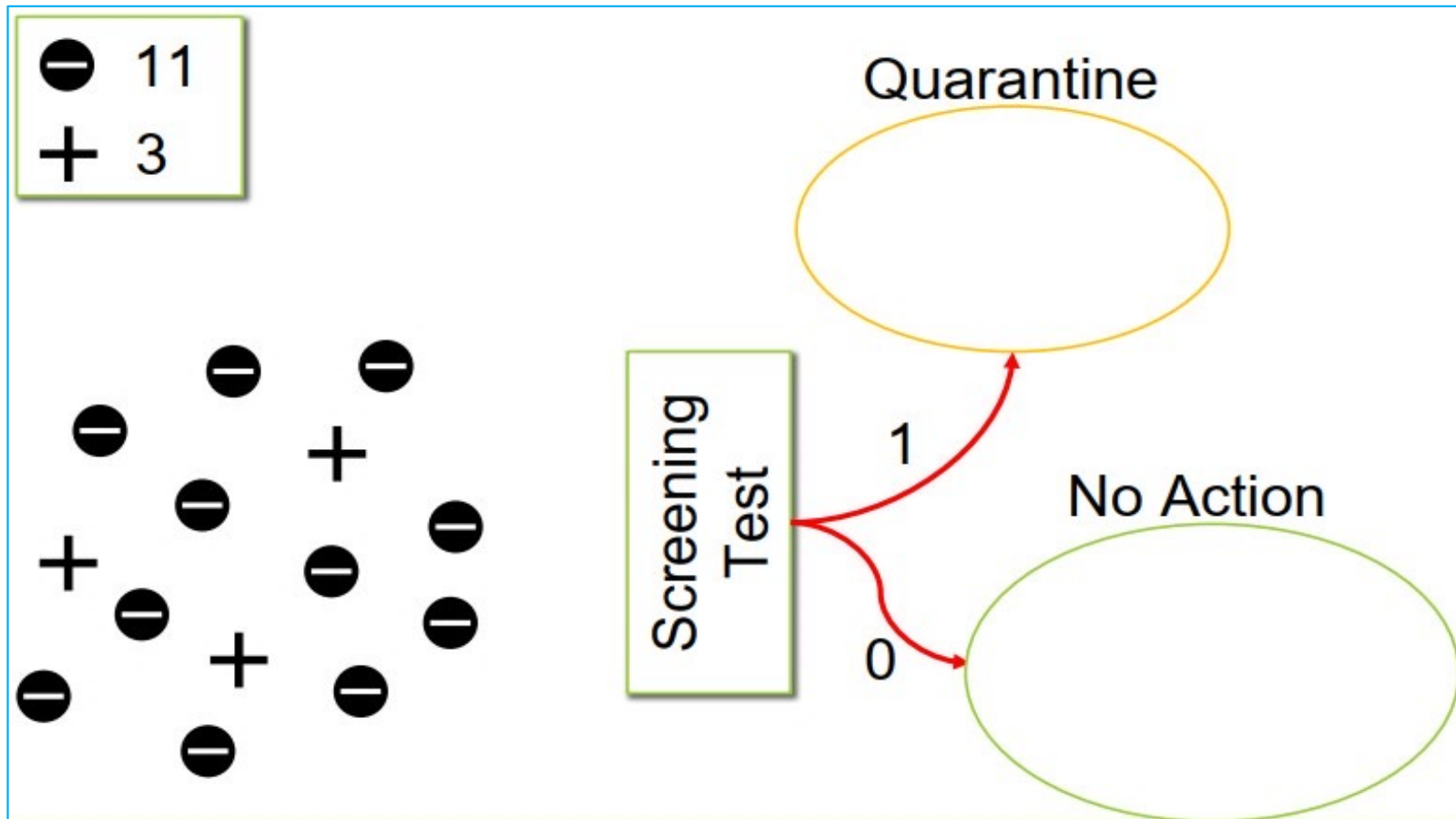
Classification Evaluation Measures

Classification evaluation metrics



- Evaluate how well the classifier $h(x)$ performs in comparison to actual classes
- With classification, different types of metrics
 - Confusion Matrix
 - Accuracy, Precision, Recall, F1-Score
 - ROC curve

Example: Screening COVID-19 @ airport

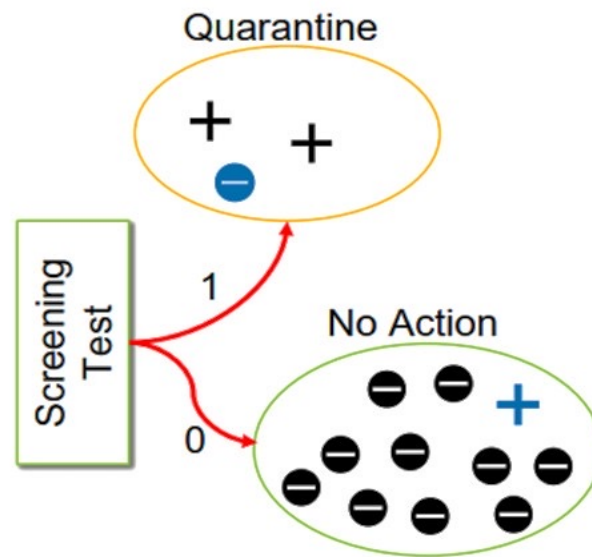


Example: Screening COVID-19 @ airport

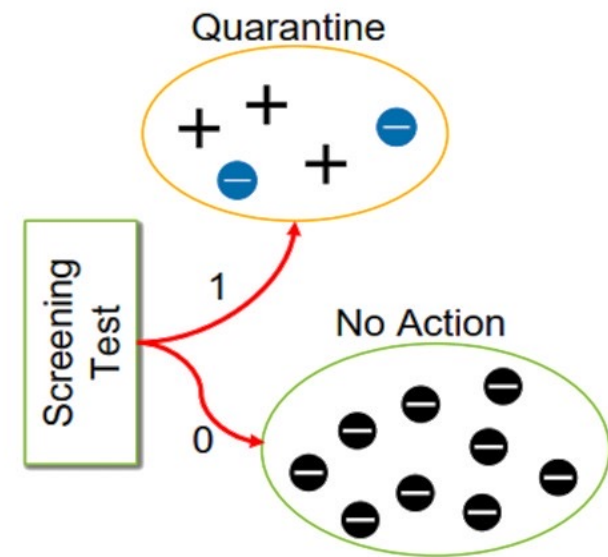
Screening done at airport:

- False Positive (Type I): **Non-COVID** patients **detected** as COVID by test.
- False Negative (Type II): **True COVID** patient **not detected** by system.

⊖	11
+	3

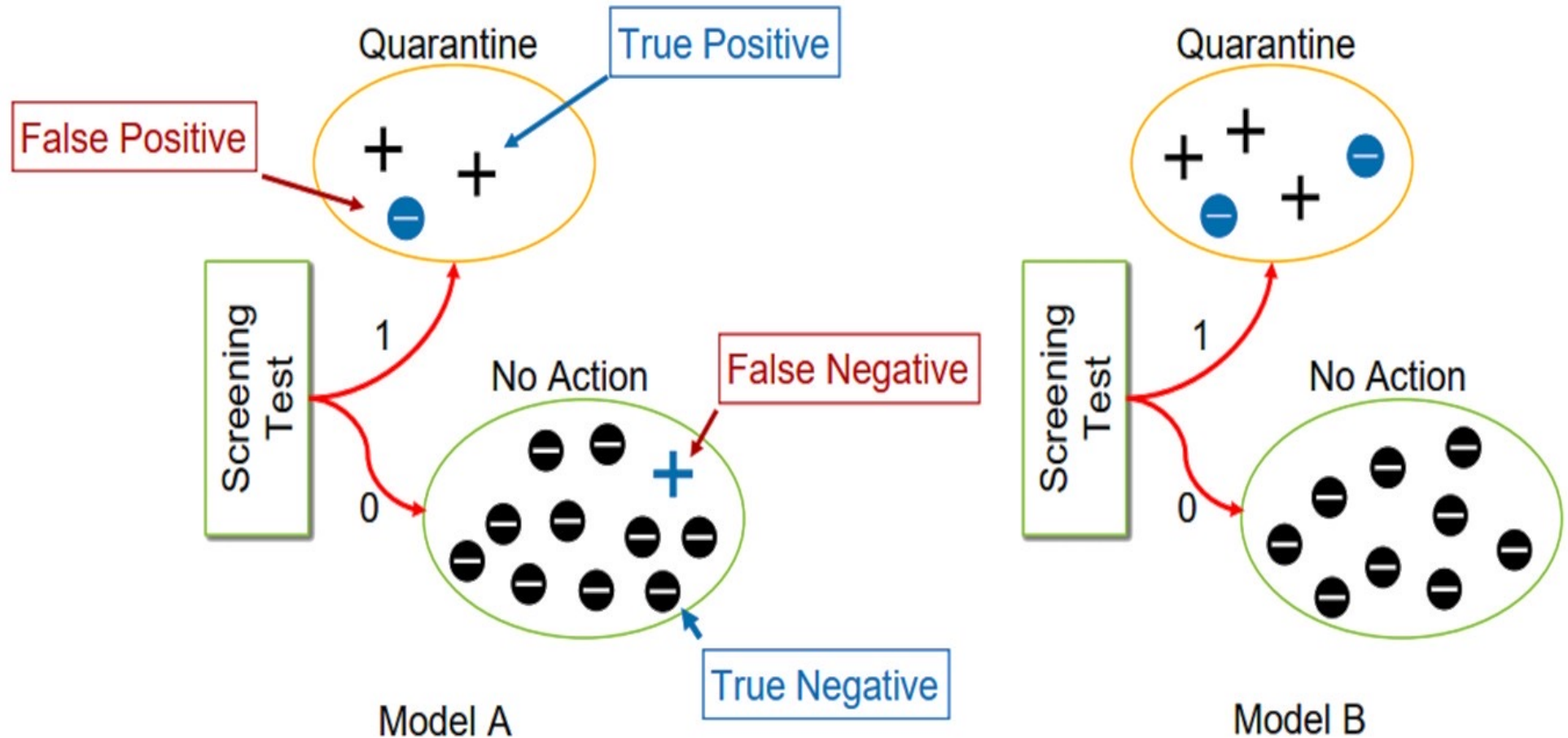


Model A



Model B

Example: Screening COVID-19 @ airport



Classification errors (Type 1 & 2)



- Consider binary class problems
- Assume there are some classification errors
 - Predict one class, but actually the test data had the other class
- Not all classification errors are the same
- From the COVID-19 example:
 - Patient does not have COVID-19 but received a positive diagnosis (Type 1 error)
 - Patient does have COVID-19 but received a negative diagnosis (Type 2 error)

Types of classification outputs

- True Positive (TP): class 1 predicted as class 1
- False Positive (FP): class 0 predicted as class 1 (type 1 error)
- True Negative (TN): class 0 predicted as class 0
- False Negative (FN): class 1 predicted as class 0 (type 2 error)

Total number of instances

$$m = TP + FP + TN + FN$$

		Prediction	
		Positive	Negative
Actual	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

Confusion matrix



- Summarise the four types of classification error

		Predicted		Y	h(X)	
		1	0			
Actual	1	TP	FN	1	1	TP
	0	FP	TN	0	0	TN
				0	0	TN
				0	1	FP
				1	1	TP
				0	0	TN
				0	1	FP
				1	0	FN
				1	1	TP

$TP = 3$
 $TN = 3$
 $FN = 1$
 $FP = 2$

		Predicted	
		1	0
Actual	1	3	1
	0	2	3

Common measures: Accuracy

Accuracy measures the rate of **correct classifications**

$$\frac{TP + TN}{TP + FP + FN + TN}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: Precision

Precision measures:

- Of all the positive instances that were identified, what ratio are actually positive?
- Is the classifier **good at being correct when it identifies a positive instance?**

$$\frac{TP}{TP + FP}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: Recall

Recall measures:

- Of all the positive instances, how many does the classifier get correct?
- Also called, the **true positive rate**, or **sensitivity**

$$\frac{TP}{TP + FN}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: Specificity



Specificity measures the “inverse” of recall

- Also known as the **true negative rate**
- Of all the actual negative instances, how many does the classifier get correct?

$$\frac{TN}{TN + FP}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: F1 Score

The **F1 score**, is the harmonic average of precision and recall

- An F1 score of 1 is perfect precision & recall
- Provides a balance between precision & recall
- More reliable measure than accuracy if the confusion matrix is skewed

$$2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Class balance



Assume that you have a binary classification problem (cancer vs healthy) where the data-set has the following distribution:

- Class 0 (Healthy) : 1000 examples
- Class 1 (Cancer) : 25 examples
- What will be the accuracy if you have a trivial model that always says healthy?

$$\text{Acc} = 1000/(1000+25) = 97.5\%$$

- Can we use this model?



ROC Curve



Plots the recall (true positive rate) against the false positive rate
A useful tool for visualising the performance of a classifier

<https://www.youtube.com/watch?v=4jRBRDbJemM>

Take home message

- Entropy and Information Gain
- Decision Tree: ID3 & C4.5 algorithms
- Pre-pruning & Post-pruning
- Bagging, and Feature Bagging
- Random Forest

Thank you!